

Rapport de projet

CyStadium : Plateforme distribuee de reservation de sieges

Fatima Aguel
Abdel El Harsal
Eleonore Georgel
Ines Ribar
Adam Terrak



3 mai 2026

Table des matières

1	Introduction	2
2	Ojectifs du sujet	2
3	Architecture globale	3
3.1	Vue logique	3
3.2	Structure du projet	4
3.3	Principes de conception	4
4	Modèles de données et persistance	5
4.1	Schema fonctionnel et entités	5
4.2	Matchs et zones tarifaires	5
4.3	Sièges (entité critique)	5
4.4	Réservations et sessions	5
4.5	Paiements	5
4.6	Persistance et intégrité des données	6
5	Analyse détaillée des modules backend	7
5.1	Communication par messages et protocole applicatif	7
5.2	Acteurs critiques	7
6	API, WebSocket et frontend	9
6.1	API REST	9
6.2	WebSocket	9
6.3	Frontend	10
7	Vérification formelle par réseaux de Petri	11
7.1	Définition du modèle réduit	11
7.2	Propriétés structurelles vérifiées	13
7.3	Propriétés structurelles	13
7.4	Propriétés LTL	13
8	Validation par tests et simulation	14
8.1	Stratégie de test	14
8.2	Scenarios critiques S1–S8 (rappel qualitatif)	14
8.3	Bilan des executions	14
9	Conclusion generale	15

1 Introduction

Notre projet CyStadium s'inscrit dans le cadre de la Coupe du Monde 2026. Il vise à concevoir une plateforme de réservation de billets pour un stade capable de gérer des flux massifs de demandes simultanées. Dans ce contexte, la fiabilité est une exigence absolue : deux clients ne doivent jamais pouvoir obtenir le même siège.

La difficulté majeure des systèmes distribués réside dans la concurrence. Sans une conception rigoureuse, des erreurs comme la double réservation ou des interblocages (deadlocks) peuvent survenir, entraînant une perte de confiance des utilisateurs et des pertes financières importantes.

Pour répondre à ces enjeux, nous avons adopté une approche hybride combinant une implémentation logicielle basée sur le modèle d'acteurs Akka et une vérification formelle via les Réseaux de Petri.

2 Ojectifs du sujet

Le projet répond à cinq objectifs pédagogiques et techniques précis :

- **État de l'art** : Étude de la vérification formelle et des réseaux de Pétri comme outils de modélisation du comportement distribué.
- **Modélisation fonctionnelle** : Définition d'une architecture basée sur les acteurs Akka pour isoler les flux critiques.
- **Traduction formelle** : Construction d'un réseau de Pétri capturant l'intégralité de l'espace d'états (*Free*, *Reserved*, *Confirmed*).
- **Vérification de propriétés** : Preuve de l'absence de deadlocks et respect des invariants métier (ex : un siège = un seul statut) via la logique temporelle linéaire (LTL).
- **Simulation comparée** : Validation de la cohérence entre le comportement observé (Scala) et le modèle théorique (Pétri).

3 Architecture globale

3.1 Vue logique

Le système est structuré en plusieurs couches fonctionnelles :

- **Frontend (Next.js/React/TypeScript)** : Interface utilisateur incluant une carte interactive des sièges mise à jour en temps réel via WebSocket.
- **API Layer (Akka HTTP)** : Exposition des routes REST et gestion de la communication directe avec le système d'acteurs via les patterns *ask/tell*.
- **Cœur Backend (Akka Classic)** :
 - **SeatActor** : Un acteur par siège individuel garantissant le traitement séquentiel des demandes (exclusivité naturelle via la mailbox Akka).
 - **SeatAllocator** : Coordonne les réservations atomiques de plusieurs sièges (protocole en deux phases : *Reserve-All* ou *Rollback-All*).
 - **ReservationHandler & PaymentGateway** : Orchestration du workflow métier et simulation de paiement asynchrone.
- **Persistance (PostgreSQL/Slick)** : Stockage transactionnel des matches, zones tarifaires et états de réservation.

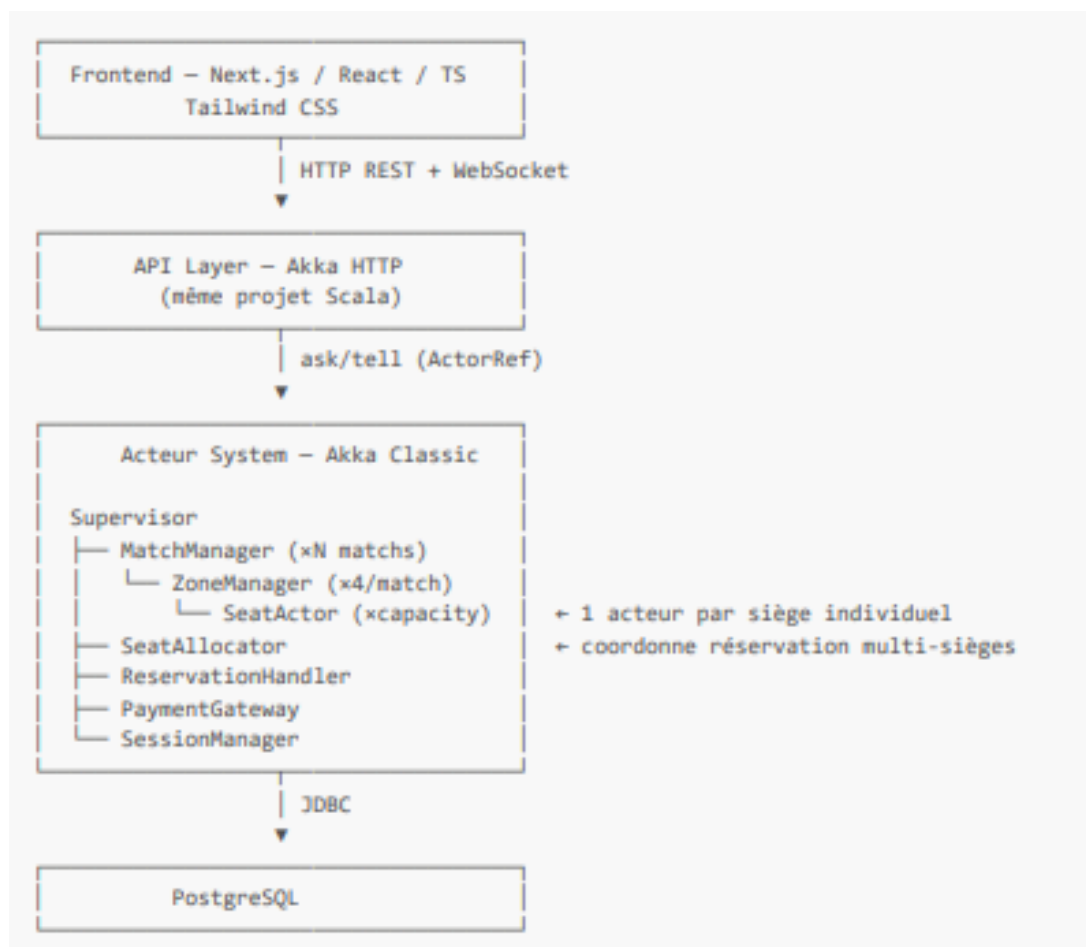


FIGURE 1 – Architecture du système.

3.2 Structure du projet

Le code source est organisé en plusieurs modules afin de séparer clairement les responsabilités et faciliter la maintenance du système :

- **src/main** : contient les fichiers Scala implémentant la logique métier, les acteurs Akka et l'API HTTP.
- **src/test** : regroupe l'ensemble des tests unitaires et d'intégration couvrant les scénarios critiques du système distribué.
- **frontend/src** : contient le code TypeScript/TSX responsable de l'interface utilisateur et des interactions en temps réel.
- **Migrations SQL** : scripts versionnés permettant l'évolution contrôlée du schéma de base de données.

3.3 Principes de conception

Le système repose sur plusieurs principes fondamentaux issus du modèle acteur et des architectures distribuées :

- **Isolement de la concurrence** : chaque *SeatActor* traite ses messages de manière séquentielle, ce qui élimine les problèmes de concurrence au niveau d'un siège individuel.
- **Coordination explicite** : le *SeatAllocator* est responsable de la gestion des réservations multi-sièges et garantit leur cohérence globale.
- **Découplage** : les protocoles de communication sont centralisés dans `Messages.scala`, ce qui permet de stabiliser les interfaces entre les différents modules.
- **Résilience** : des mécanismes de supervision et de timeout sont appliqués sur les workflows critiques afin de garantir la robustesse du système en cas de panne ou d'erreur.

4 Modèles de données et persistance

4.1 Schema fonctionnel et entités

Le modèle de données repose sur une hiérarchie stricte permettant de gérer la tarification et la disponibilité pour les 64 matchs de la Coupe du Monde.

4.2 Matches et zones tarifaires

Chaque match est découpé en quatre zones distinctes avec des tarifs fixes :

- **VIP** : 500€
- **Or** : 250€
- **Standard** : 100€
- **Populaire** : 50€

Une zone contient un ensemble de sièges individuels identifiés par un label unique (ex : *VIP-A-042*).

4.3 Sièges (entité critique)

Chaque siège possède les attributs techniques suivants :

- **id** : UUID
- **zoneId** : identifiant de la zone
- **label** : identifiant lisible du siège
- **row** : rangée
- **number** : numéro du siège

Machine à états : un siège peut uniquement se trouver dans l'un des quatre états suivants :

- *Free*
- *Reserved*
- *Confirmed*
- *Locked* (pendant une transaction en cours)

4.4 Réservations et sessions

Une réservation regroupe un ou plusieurs sièges d'une même zone pour un match donné.

Contrainte temporelle : la réservation est temporaire. Elle expire automatiquement après 10 minutes si le paiement n'est pas validé, entraînant la libération immédiate des sièges.

Les sessions clients ont une durée de vie liée à l'inactivité, avec une expiration après 15 minutes.

4.5 Paiements

Chaque tentative de paiement est tracée avec les statuts suivants :

- *pending*
- *success*
- *failed*
- *timeout*

Le système simule des comportements réalistes afin de tester la robustesse des mécanismes de rollback :

- 80% de succès
- 10% d'échec (carte refusée)
- 10% de timeout

4.6 Persistance et intégrité des données

La persistance du système CyStadium ne se limite pas à un simple stockage des données. Elle joue un rôle essentiel pour garantir l'intégrité du système, notamment en cas d'accès concurrents.

La base de données évolue grâce à des migrations SQL versionnées (de V1 à V5), ce qui permet de modifier progressivement le schéma tout en gardant un bon contrôle des changements.

La cohérence globale du système repose sur trois éléments principaux.

Contraintes d'intégrité Tout d'abord, la fiabilité des données est assurée par des contraintes relationnelles strictes. En plus des clés étrangères (FK) reliant les sièges aux zones et les réservations aux clients, des contraintes **CHECK** sont utilisées pour contrôler certaines valeurs directement en base.

Par exemple, le statut d'un siège est limité aux valeurs suivantes : *free*, *reserved*, *confirmed*, *locked*. Cela permet d'éviter toute incohérence, même si une erreur survient dans la logique applicative.

Des index sont également mis en place, notamment sur le couple (**zone_id**, **status**), afin d'améliorer les performances lors des recherches de sièges disponibles en temps réel.

Atomicité des opérations Ensuite, l'atomicité est garantie grâce à l'utilisation des transactions via DBIO (Slick).

Même si les acteurs Akka traitent les messages de manière séquentielle, certaines opérations restent complexes. Par exemple, lorsqu'un utilisateur réserve plusieurs sièges, il faut à la fois créer une réservation et verrouiller tous les sièges concernés.

Ces actions sont donc regroupées dans une transaction. Ainsi, en cas d'erreur (crash, problème réseau, etc.), aucune modification partielle n'est conservée. Cela évite par exemple qu'un siège soit marqué comme réservé sans qu'une réservation existe réellement.

Synchronisation des états Enfin, le système garantit une synchronisation stricte entre les états en mémoire (côté acteurs) et ceux enregistrés en base de données.

Lorsqu'une réservation est validée, plusieurs éléments sont mis à jour en même temps : le paiement passe à *success*, la réservation à *confirmed*, et les sièges associés sont également confirmés.

Cette cohérence permet notamment au *MatchManager* de restaurer correctement l'état du système après un redémarrage, ce qui renforce la robustesse et la résilience de la plateforme.

5 Analyse détaillée des modules backend

5.1 Communication par messages et protocole applicatif

Dans une architecture distribuée basée sur Akka, la communication entre les différents composants repose exclusivement sur l'échange de messages asynchrones.

Afin de structurer ces échanges, toutes les définitions ont été centralisées dans le fichier `protocol/Messages.scala`. Ce fichier ne correspond pas uniquement à une liste de messages : il représente un véritable contrat de communication entre les différentes parties du projet (backend, API et frontend).

L'utilisation de *sealed traits* et de *case classes* en Scala permet de définir des protocoles typés, ce qui rend les échanges plus sûrs et plus faciles à maintenir. Chaque grande fonctionnalité du système est ainsi associée à un ensemble de messages spécifiques.

Gestion des accès Les messages `Login` et `ValidateSession` permettent de gérer l'authentification et la validation des sessions utilisateurs.

Flux de disponibilité Les requêtes `CheckAvailability` sont utilisées pour récupérer l'état des sièges en temps réel, notamment via les gestionnaires de zones.

Cœur transactionnel Le protocole encadre les opérations critiques du système, comme la réservation d'un siège (`ReserveSeat`) ou la gestion de réservations multiples via le *SeatAllocator*.

Finalisation des réservations Les messages liés au paiement (`InitPayment`, `PaymentSuccess`) ainsi que ceux de confirmation (`ConfirmReservation`) permettent de finaliser le processus d'achat de manière sécurisée.

Choix méthodologique La centralisation des messages a été un choix important pour la stabilité du projet. En définissant les interfaces dès le début du développement, nous avons limité les problèmes lors de l'intégration des différents modules.

Chaque membre de l'équipe a ainsi pu travailler de manière indépendante sur ses composants, tout en respectant un contrat commun. Cela a également permis de stabiliser les échanges au niveau de l'API REST et des WebSockets.

5.2 Acteurs critiques

`SeatActor` / `ZoneManager` / `MatchManager`

Ces modules gèrent la granularité de l'état des sièges et l'agrégation par zone/match. Le `SeatActor` : chaque siège individuel est piloté par un acteur dédié. Chaque acteur traite ses messages séquentiellement via sa mailbox, garantissant qu'aucune race condition n'est possible. Son état interne peut être défini comme suit :

- **Free** : Disponible à la vente.
- **Reserved** : Bloqué temporairement en attente de paiement (timeout de 10 minutes).
- **Confirmed** : Définitivement vendu après succès du paiement.
- **Locked** : Verrouillé pendant une opération technique.

Lorsqu'un client effectue une demande de réservation, le *SeatActor* vérifie son état courant. Si celui-ci est *Free*, il passe à l'état *Reserved*. En revanche, si une seconde demande intervient durant cette période, elle est automatiquement rejetée avec une réponse de type *SeatUnavailable*.

SeatAllocator

Ce module est responsable de la réservation multi-sièges. Il implémente un protocole de coordination en deux phases pour garantir qu'un client n'obtienne pas seulement une partie de sa commande :

Phase de réservation Le système envoie une requête `ReserveSeat` à l'ensemble des *SeatActor* ciblés.

Phase de décision Si tous les sièges répondent favorablement, la réservation est validée. En revanche, si un seul siège renvoie une réponse `SeatUnavailable`, le module déclenche immédiatement un rollback en envoyant un message `ReleaseSeat` aux acteurs déjà sollicités.

Ce mécanisme garantit la propriété d'anti-surbooking du système, en assurant qu'aucune réservation partielle ne puisse être validée.

ReservationHandler

Il agit comme le chef d'orchestre du workflow métier. Son rôle est de centraliser la logique de haut niveau : validation de la session utilisateur auprès du `SessionManager`, création de l'entrée en base de données, et gestion de l'expiration TTL (Time To Live). Il assure la transition des états de la réservation globale.

L'état fonctionnel couvert est :

Pending → Paid → Confirmed ou Pending → Cancelled.

PaymentGateway

Simulation des paiements asynchrones Afin de simuler un environnement réaliste, ce module traite les paiements de manière asynchrone en appliquant une répartition statistique des résultats :

- 80% de succès
- 10% d'échecs (refus bancaire)
- 10% de timeouts

L'utilisation de la méthode `scheduleOnce` d'Akka est essentielle dans ce contexte. Elle permet de simuler le délai de réponse d'un service bancaire externe de manière non bloquante.

Ainsi, aucun thread du système d'acteurs n'est immobilisé pendant l'attente du résultat, ce qui garantit de bonnes performances et respecte les principes du modèle acteur.

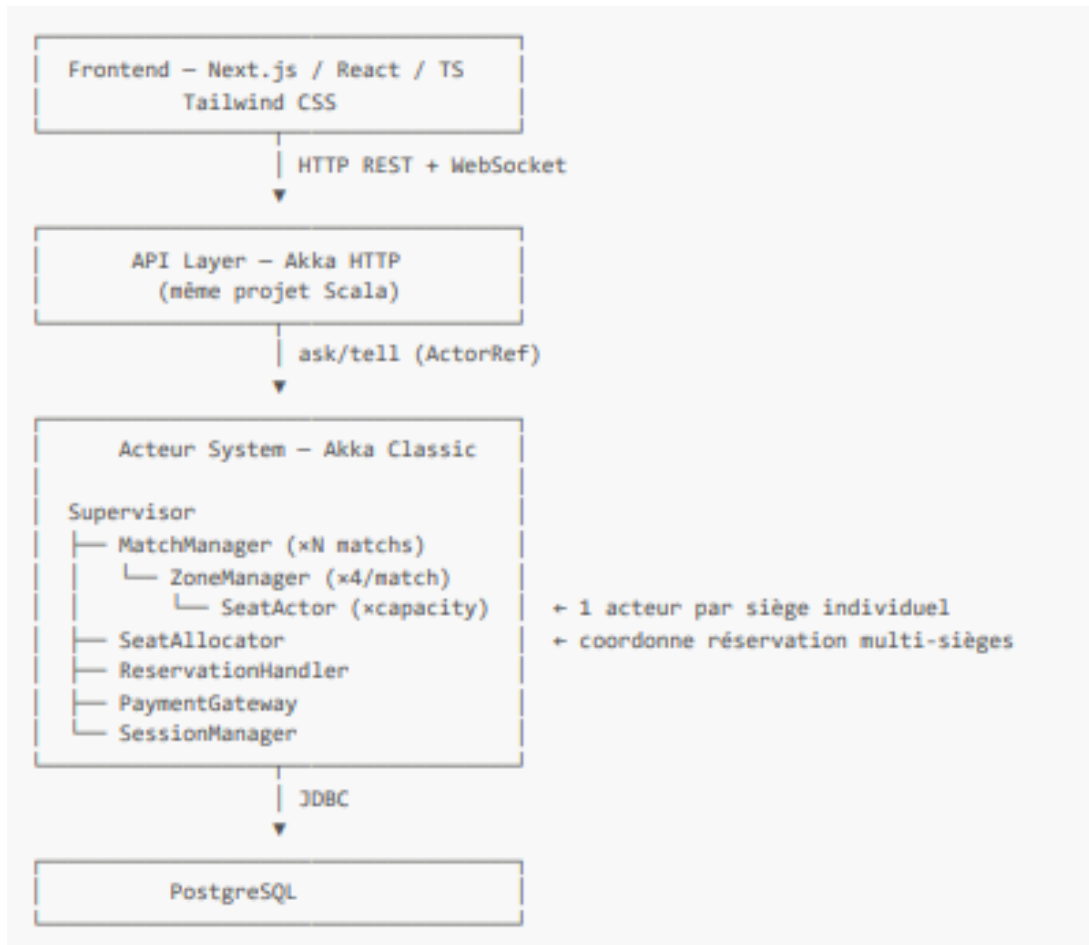


FIGURE 2 – Hiérarchie des Acteurs CyStadium

6 API, WebSocket et frontend

6.1 API REST

Les routes couvrent :

- authentification et session (`/api/auth/...`);
- consultation des matchs, zones et sièges;
- creation, paiement, annulation et consultation des réservations;
- endpoints admin : statistiques, utilisateurs, matchs, réservations, monitoring des acteurs.

Les réponses d'erreur typiques suivent le contrat groupe : par exemple 503 en cas de timeout acteur, 401 si la session est invalide. Les payloads JSON utilisent la convention `snake_case` définie dans `json/Codecs.scala`.

6.2 WebSocket

Deux usages principaux :

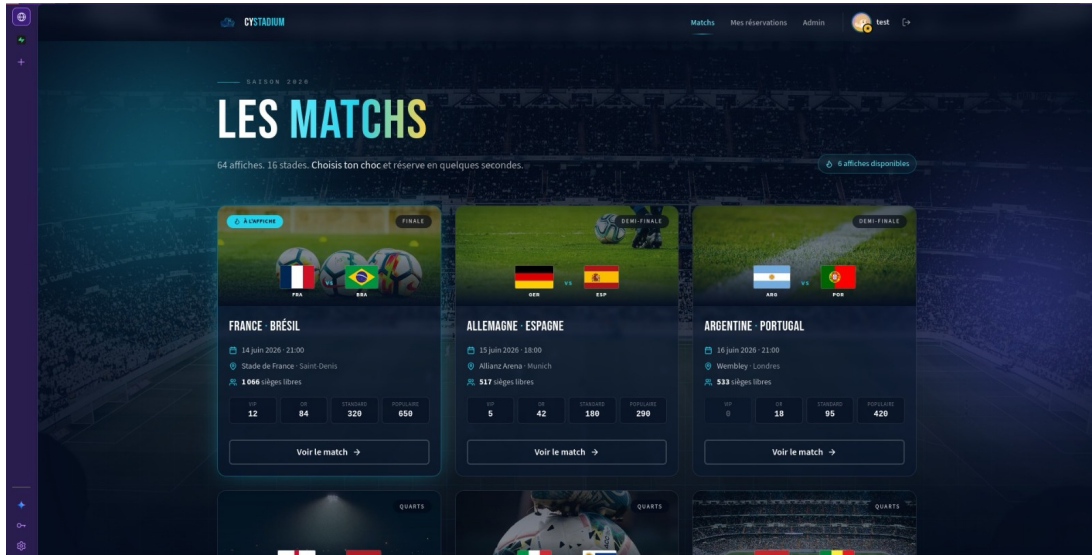
- diffusion des changements de statut de sièges vers les clients (`SeatStatusEvent` sur le topic du match);
- monitoring temps réel pour l'administration (flux dédié au suivi de l'actor system).

Cette séparation permet une démo visuelle forte lors de la soutenance (concurrency observable sur la carte des sièges).

6.3 Frontend

Le frontend Next.js fournit les ecrans metier attendus : liste des matchs, selection de sieges, panier, paiement simule, historique des reservations, dashboard admin. Le composant de carte de sieges (**SeatMap**) est central pour visualiser les effets de la concurrence et valider le comportement attendu cote utilisateur.

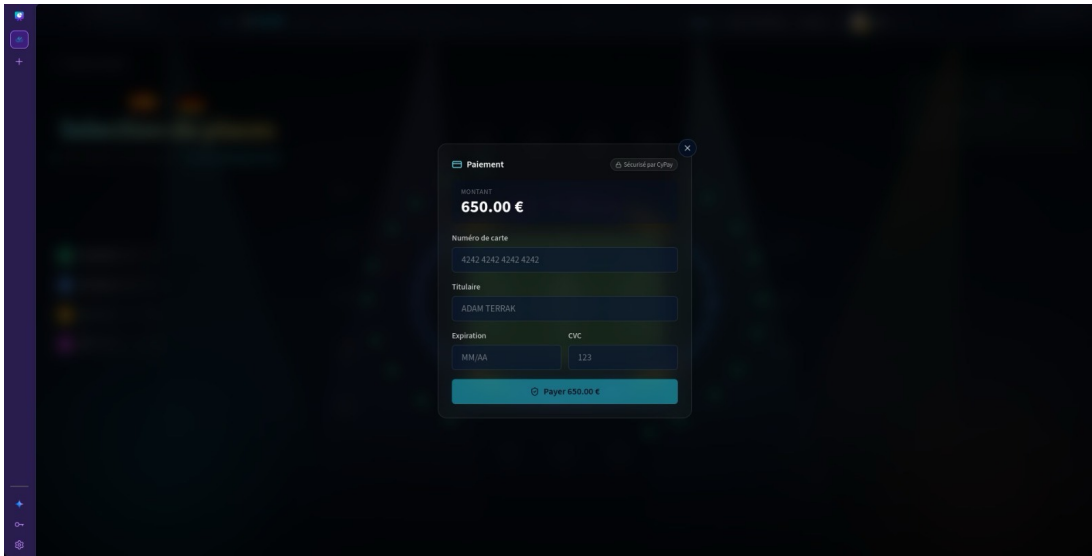
1. Écran d'accueil ou liste des matchs : cartes ou lignes avec nom des équipes, date, stade, lien ou bouton *Réserver*.



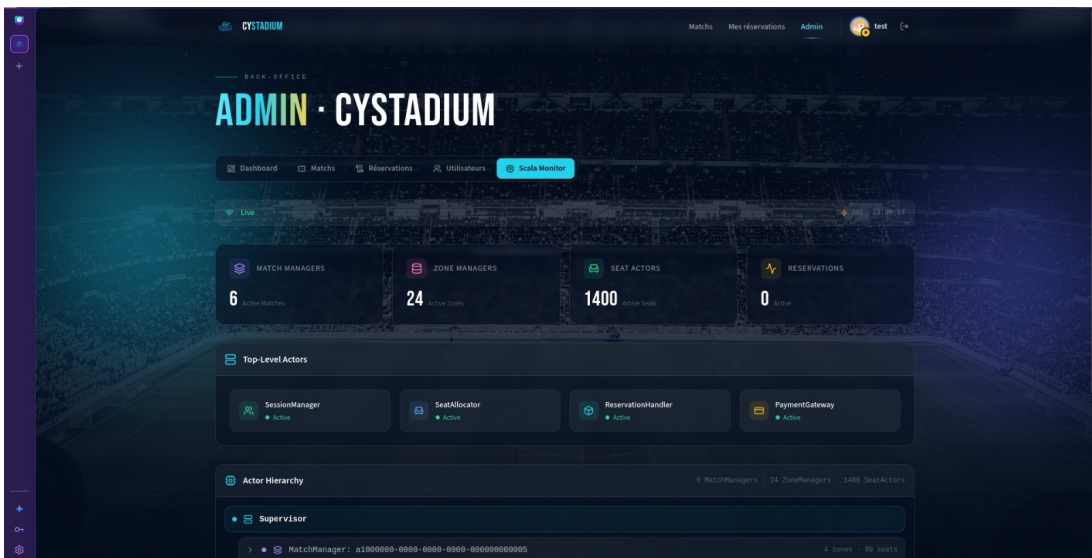
2. Carte des sièges pour un match : grille ou plan de zone avec codes (ex. A1, A2), légende des couleurs.



3. Étape panier ou paiement simulé : résumé des sièges choisis, montant total, bouton de validation ou message de succès/échec.



4. Dashboard administrateur : vue synthétique (statistiques, liste réservations) et panneau de monitoring temps réel.



7 Vérification formelle par réseaux de Petri

Le réseau de Petri est un modèle mathématique permettant de simuler de manière graphique et analytique le flux d'un système distribué. Dans le cadre de ce projet, nous avons développé un analyseur spécifique en Scala afin d'explorer l'ensemble des chemins possibles du système.

7.1 Définition du modèle réduit

Pour l'analyse, un modèle simplifié a été utilisé : un match, une zone, trois sièges (A1, A2, A3) et deux clients concurrents.

Places (états)

- SeatFree
- SeatReserved
- SeatConfirmed

- ClientActive
- WaitingForPayment

Transitions (actions)

- Reserve
- PaySuccess
- PayFail
- Confirm
- Release

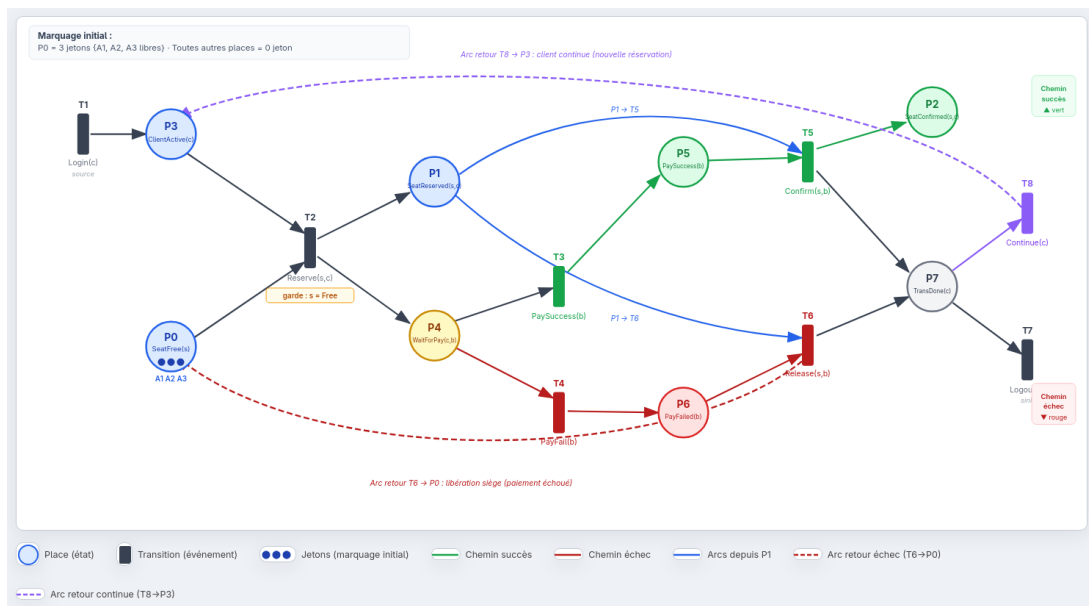


FIGURE 3 – Réseau de Petri CyStadium

7.2 Propriétés structurelles vérifiées

L'analyseur a permis de vérifier les propriétés suivantes :

- **Bornitude** : le nombre de jetons dans le système est fini et correspond à la capacité physique du stade.
- **Vivacité** : aucune situation bloquante n'existe ; toutes les actions prévues dans le modèle peuvent être déclenchées.
- **Absence de deadlock** : il existe toujours un chemin permettant d'atteindre un état terminal, que ce soit un paiement réussi ou une annulation.

7.3 Propriétés structurelles

L'analyseur calcule notamment :

- bornitude ;
- absence de deadlock (ou présence de deadlock inattendu selon le modèle) ;
- vivacité ;
- vérification d'invariants métier sur l'ensemble des marquages accessibles.

7.4 Propriétés LTL

Les propriétés métier cibles sont encodées :

- **S1** : absence de double réservation ;
- **S2** : un siège possède exactement un statut ;
- **S3** : un siège confirmé implique un paiement valide ;
- **S4** : un échec de paiement implique une libération future ;
- **V1/V2** : toute attente de paiement se termine ;
- **V3** : absence de deadlock inattendu.

8 Validation par tests et simulation

8.1 Stratégie de test

#	Scénario	Acteurs impliqués
S1	1 client réserve et paie	ReservationHandler, SeatAllocator, PaymentGateway
S2	Paieement refusé → rollback	PaymentGateway, SeatAllocator
S3	Timeout de 10 min → libération	ReservationHandler (timer)
S4	2 clients réservent le même siège simultanément	SeatAllocator → SeatActor (condition de concurrence)
S5	2 clients réservent des sièges différents	SeatAllocator (exécution parallèle)
S6	1 client réserve 3 sièges (tout-ou-rien)	SeatAllocator (protocole en deux phases)
S7	Crash du ZoneManager → restauration	MatchManager (supervision)
S8	3 clients pour 2 sièges disponibles	SeatAllocator (anti-surbooking)

TABLE 1 – Scénarios de tests réalisés

8.2 Scenarios critiques S1–S8 (rappel qualitatif)

Les simulations couvrent notamment :

- cas nominal (reservation + paiement OK) ;
- paiement refuse et timeout (rollback / liberation) ;
- courses critiques sur le meme siege ;
- reservations paralleles sur des sieges differents ;
- reservation atomique multi-sieges ;
- anti-surbooking multi-clients ;
- supervision / restauration (S7).

8.3 Bilan des executions

Selon la campagne de reference :

- 52 tests Scala executes, 52/52 en succes ;
- build backend et frontend valides ;
- couverture des scenarios critiques (reservation simple, courses concurrentes, timeout, anti-surbooking multi-clients, supervision).

Scenario	Description	Statut
S1	Un client reserve puis confirme	OK
S2	Echec partiel multi-sieges et rollback	OK
S3	Expiration TTL et liberation automatique	OK
S4	Deux clients en concurrence sur le meme siege	OK
S5	Reservations paralleles sur sieges differents	OK
S6	Reservation atomique de trois sieges	OK

S7	Restauration/supervision d'acteur	OK
S8	Trois clients pour deux sieges disponibles	OK

Chronogrammes de simulation des scenarios S1, S4 et S8 6.5cm A inserer : timeline des evenements (request, reserve, pay, confirm/release).

Fichier propose : `livrables/figures/simulation-timelines.pdf`

9 Conclusion generale

Le projet CyStadium satisfait l'essentiel des exigences techniques et scientifiques du sujet. L'architecture acteur traite correctement la concurrence métier, la simulation de paiement est non bloquante, et la vérification formelle apporte une couche de confiance supplémentaire sur les invariants critiques. Avec la finalisation documentaire (rapport/bibliographie/CI), le projet est en position solide pour la soutenance.

Annexe A – Commandes utiles

- Compilation/tests backend : `sbt test`
- Build frontend : `npm run build` (dans frontend/)
- Lancer la stack : `docker compose up -build -d`
- Logs backend : `docker compose logs -f backend`
- Logs frontend : `docker compose logs -f frontend`

Annexe B – Références bibliographiques minimales (à compléter)

- T. Murata, *Petri Nets : Properties, Analysis and Applications*, Proceedings of the IEEE, 1989.
- E. Clarke, O. Grumberg, D. Peled, *Model Checking*, MIT Press, 1999.
- C. Baier, J.-P. Katoen, *Principles of Model Checking*, MIT Press, 2008.
- A. Pnueli, *The Temporal Logic of Programs*, FOCS, 1977.
- Documentation officielle Akka : <https://doc.akka.io/docs/akka/current/>.